

姓名： 张洋 学号： 20232411

《卫星导航定位基础》课后作业（一）

利用广播星历计算 BDS 卫星的位置

任务部分：每位同学登陆网站（ftp://epncb.oma.be/pub/obs/BRDC/2025/或ftp://pub:tarc@ftp2.csno-tarc.cn/almanac/2025）下载一个不同天的广播星历文件，批量计算一天所有 BDS 卫星的位置、高度角和方位角，并绘制二维和三维分布图，并与精密星历进行对比评价轨道精度。

一、 算法

北斗广播星历由 6 个开普勒轨道根数、3 个摄动线性改正项和 6 个周期改正项系数 等一共 15 个描述轨道特征的参数和 1 个星历参考时间组成，其广播星历参数的更新时间是 1h。

序号	参数	定义
1	t_{oe}	星历参考时间
2	$\sqrt{A}$	卫星轨道长半轴的平方根
3	e	轨道偏心率
4	i_0	t_{oe} 时的轨道倾角
5	Ω_0	周历元零时刻计算的升交点经度
6	ω	轨道近地点角距
7	M_0	t_{oe} 时的平近点角
8	Δn	平均运动角速率改正值
9	$\dot{\Omega}$	升交点赤经变化率
10	$\dot{i}$	轨道倾角变化率
11	C_{uc}	升交点角距余弦调和改正项振幅
12	C_{us}	升交点角距正弦调和改正项振幅
13	C_{rc}	卫星地心距余弦调和改正项振幅
14	C_{rs}	卫星地心距正弦调和改正项振幅
15	C_{ic}	轨道倾角余弦调和改正项振幅
16	C_{is}	轨道倾角正弦调和改正项振幅

在北斗二号广播星历的 16 个参数中, t_{oe} 表示星历参考时间, 从每星期六/星期日子夜零点起算, 即星期六的 24:00:00 或者星期日的 00:00:00 起算。其余 15 个参数表示的轨道运动情况如图 4.11 所示。

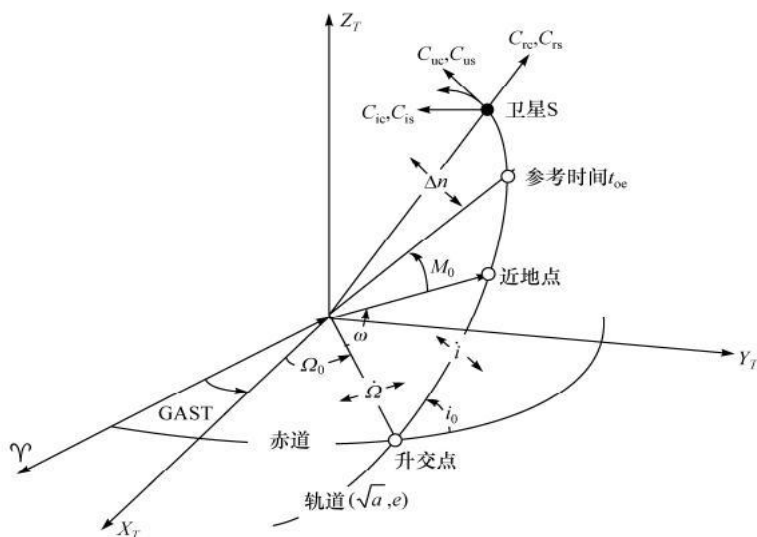


图 4.11 北斗二号广播星历示意图

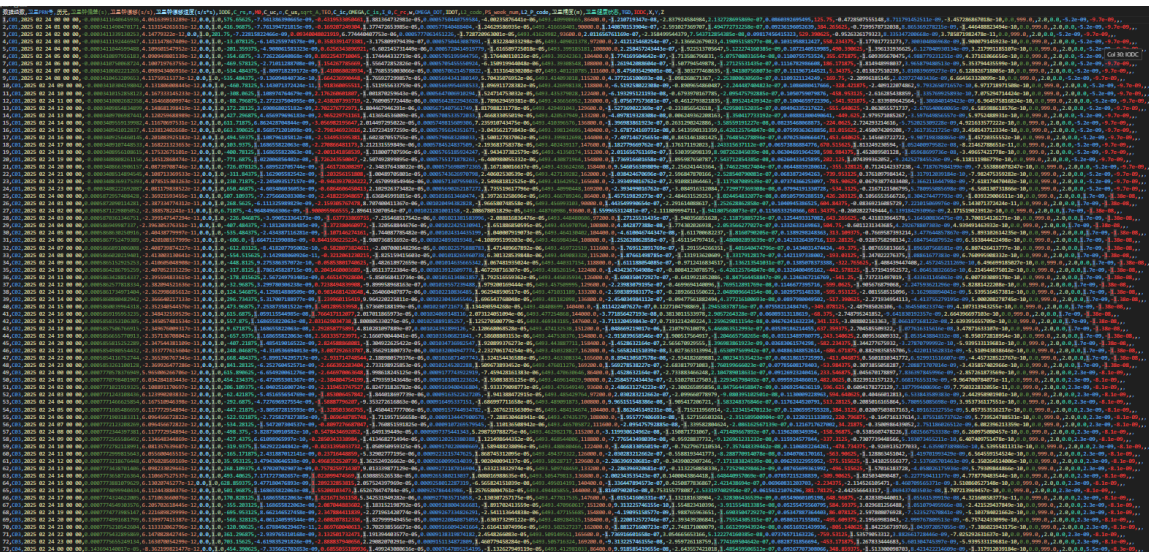
首先, 对 2 月 24 日广播星历进行预处理, 数据预处理步骤如下:

提取北斗卫星数据: 读取广播星历文件, 找到数据块起始位置, 筛选出以 'C' 开头的北斗卫星数据行, 按 8 行一组收集, 存储到新文件 all_C.txt。如下图所示:

```
C01 2025 02 24 00 00 00-3.411640645936e-04 4.061639913289e-12 0.000000000000e+00
1.000000000000e+00 5.756562500000e+02-7.561386390665e-09-4.319533050461e-01
1.881364732981e-05 5.750440759584e-04-4.002358764410e-06 6.493409900665e+03
8.640000000000e+04-1.210719347000e-08-2.837924584984e+00 2.132728695869e-07
8.603926954950e-02 1.257500000000e+02-4.728507555148e-01 8.711791452511e-09
-3.457286867018e-10 0.000000000000e+00 9.990000000000e+02 0.000000000000e+00
2.000000000000e+00 0.000000000000e+00-5.200000000000e-09-9.700000000000e-09
8.640000000000e+04 0.000000000000e+00
C01 2025 02 24 01 00 00-3.411490470171e-04 4.113154261631e-12 0.000000000000e+00
1.000000000000e+00 4.169687500000e+02-7.761394721815e-09-1.692072493040e-01
1.377472653985e-05 5.778404884040e-04-1.246295869350e-05 6.493416658401e+03
9.000000000000e+04 1.140870153904e-07-2.591027369707e+00 1.494772732258e-07
9.236196052639e-02 3.842656250000e+02-7.199578732038e-01 8.865369278215e-09
-1.446488823494e-10 0.000000000000e+00 9.990000000000e+02 0.000000000000e+00
2.000000000000e+00 0.000000000000e+00-5.200000000000e-09-9.700000000000e-09
9.000000000000e+04 0.000000000000e+00
```

解析数据并存为字典: 读取 all_C.txt, 确定数据起始行, 计算数据块个数。逐组解析数据, 将每行特定位置数据对应卫星的不同参

数，存入字典，所有字典构成列表。最后将列表数据写入北斗卫星.csv 文件。如下图所示：



接着，读取上述 csv 文件中的 2 月 24 日的星历参数，进行北斗卫星坐标位置的计算。如下图所示：

参数的计算公式	公式的含义
$\mu = 3.986004418 \times 10^{14} \text{ m}^3 / \text{s}^2$	CGCS2000 坐标系下的地球引力常数
$\dot{\Omega}_e = 7.292115 \times 10^{-5} \text{ rad} / \text{s}$	CGCS2000 坐标系下的地球自转速率
$\pi = 3.1415926535898$	圆周率
$A = (\sqrt{A})^2$	长半轴的计算
$n_0 = \sqrt{\frac{\mu}{A^3}}$	卫星平均运动角速率的计算
$t_k = t - t_{oc}$	观测历元到参考历元的时间差的计算
$n = n_0 + \Delta n$	平均运动角速率的改正
$M_k = M_0 + nt_k$	平近点角的计算
$E_k = M_k + e \sin E_k$	偏近点角的迭代计算
$\begin{cases} \sin v_k = \frac{\sqrt{1-e^2} \sin E_k}{1-e \cos E_k} \\ \cos v_k = \frac{\cos E_k - e}{1-e \cos E_k} \end{cases}$	真近点角的计算
$\phi_k = v_k + \omega$	纬度幅角的计算
$\begin{cases} \delta u_k = C_{us} \sin(2\phi_k) + C_{uc} \cos(2\phi_k) \\ \delta r_k = C_{ns} \sin(2\phi_k) + C_{nc} \cos(2\phi_k) \\ \delta i_k = C_{is} \sin(2\phi_k) + C_{ic} \cos(2\phi_k) \end{cases}$	纬度幅角改正项 地心距改正项 轨道倾角改正项

$u_k = \phi_k + \delta u_k$	计算改正后的纬度幅角
$r_k = A(1 - e \cos E_k) + \delta r_k$	计算改正后的地心距
$i_k = i_0 + \dot{i} \cdot t_k + \delta i_k$	计算改正后的倾角
$\begin{cases} x_k = r_k \cos u_k \\ y_k = r_k \sin u_k \end{cases}$	计算卫星在轨道平面内的坐标
$\Omega_k = \Omega_0 + (\dot{\Omega} - \dot{\Omega}_e)t_k - \dot{\Omega}_e t_{oe}$ $\begin{cases} X_k = x_k \cos \Omega_k - y_k \cos i_k \sin \Omega_k \\ Y_k = x_k \sin \Omega_k + y_k \cos i_k \cos \Omega_k \\ Z_k = y_k \sin i_k \end{cases}$	对于 MEO/IGSO 卫星： 1) 计算历元升交点的经度 2) 计算 MEO/IGSO 卫星在 CGCS2000 坐标系中的坐标
$\Omega_k = \Omega_0 + \dot{\Omega} t_k - \dot{\Omega}_e t_{oe}$ $\begin{cases} X_{GK} = x_k \cos \Omega_k - y_k \cos i_k \sin \Omega_k \\ Y_{GK} = x_k \sin \Omega_k + y_k \cos i_k \cos \Omega_k \\ Z_{GK} = y_k \sin i_k \end{cases}$ $\begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix} = R_z(\dot{\Omega} t_k) R_x(-5^\circ) \begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix}$	对于 GEO 卫星： 1) 计算历元升交点的经度 2) 计算 GEO 卫星在自定义坐标系中的坐标 3) 计算 GEO 卫星在 CGCS2000 坐标系中的坐标

t 是信号发射时刻的北斗时。 t_k 是信号发射时刻 t 和星历参考时间 t_{oe} 之间的时间差，并考虑到跨过一周开始或结束的时间，其计算公式为：（时间差是由星历得到的信号传播时间差吗？星历是地面监控站接收到的卫星发出来我们接受到的星历）。

$$t_k = \begin{cases} t_k - 604800, & t_k > 302400 \\ t_k + 604800, & t_k < -302400 \end{cases}$$

计算得出北斗卫星的坐标位置，输出为文件 北斗卫星位置.txt。根据该文件的数据绘制 2D、3D 图来可视化展示卫星位置。

然后，对 2 月 24 日精密星历数据进行预处理。筛选出北斗卫星的整小时的精密星历数据并按照卫星序号归类，输出为新文件 sp3_data.txt。如图：

```
C06, -7287.700211, 23584.105371, 34285.241577
C06, -11800.726419, 23889.294153, 32724.569533
C06, -15553.428942, 26420.081564, 28927.322676
C06, -17531.273909, 30489.169672, 23146.242294
C06, -17200.156995, 34991.532500, 15772.467237
C06, -14652.793918, 38702.091973, 7310.101672
C06, -10585.640273, 40607.890868, -1658.006876
```


继而，对 sp3_data.txt 的 2 月 24 日数据真值和 北斗卫星位置.txt 的 2 月 24 日数据计算值进行误差计算，百分差计算：

误差：|真值 - 计算值|；

百分差：|真值 - 计算值| / 真值；

最后，计算卫星数据计算的平均误差和平均百分差，作为整个程序的精度指标，绘制误差散点图和箱线图对误差分布进行评估。

二、 代码

运行环境：python 3.13.2；

库依赖：math, pandas, numpy, os, csv, matplotlib, plotly；

星历时间：2025 年 2 月 24 日；

代码结构：

- main.py - 解析广播星历文件，数据预处理程序；
- draw2D.py - 绘制卫星位置的 2D 图像；
- draw3D.py - 绘制卫星位置的 3D 图像；
- sel6-45.py - 筛选出计算值中卫星编号为 C06-C45 的数据；
- sp3.py - 解析精密星历文件，数据预处理程序；
- ana.py - 计算误差与百分差；
- config.py - 计算所有卫星的平均误差和平均百分差；
- plot.py - 绘制计算误差的箱线图；
- plot1.py - 绘制计算误差的分布散点图；

具体代码如下：

main.py:

```
import math as m
import numpy as np
import csv
import os

def rx(fai):
    result = np.asmatrix([[1, 0, 0], [0, m.cos(fai), m.sin(fai)],
[0, -1 * m.sin(fai), m.cos(fai)]])
```

```

    return result

def rz(fai):
    result = np.asmatrix([[m.cos(fai), m.sin(fai), 0], [-1 *
m.sin(fai), m.cos(fai), 0], [0, 0, 1]])
    return result

def reader_GNSS(filename):
    infile = open("BRDC00GOP_R_20250550000_01D_MN.rnx", "r")
    # 读取原星历文件
    content = infile.readlines() # 按行读取文件
    infile.close() # 关闭文件
    start_num = 0
    all_C = [] # 存储北斗星历数据块

    for i in range(len(content)): # len(content)为文件的行数
        if
content[i].find("
END OF HEADER") != -1:
            start_num = i + 1 # 找到数据块开始的位置
        processed_lines = set() # 用于记录已经处理过的行号
        for i in range(len(content) - start_num): # 遍历数据块
            line = content[start_num + i] # 第i行
            if line[0] == 'C':
                for j in range(0, 8):
                    line_index = start_num + i + j
                    if line_index < len(content) and line_index not
in processed_lines:
                        all_C.append(content[line_index])
                        processed_lines.add(line_index)
        outFile = open("all_C.txt", 'w') # 存储为新的文件（只含有北斗卫
星）
        outFile.writelines(all_C)
        outFile.close()

# 接下来是计算卫星位置的程序
with open('all_C.txt', 'r') as f:
    if f == 0: print("不能打开文件！")
    else: print("导航文件打开成功！")
    nfile_lines = f.readlines() # 按行读取N文件
    print(len(nfile_lines)) # 打印N文件的行数 9216
    f.close()

```

```

start_num = 0
for i in range(len(nfile_lines)): # 遍历N文件
    if nfile_lines[i].find('END OF HEADER') != -1:
        start_num = i + 1

n_dic_list = [] # 存储字典

n_data_lines_nums = int((len(nfile_lines) - start_num) / 8) #
计算数据块的个数
print("一共%d组数据" % (n_data_lines_nums))

# 第j组, 第i行
for j in range(n_data_lines_nums):
    n_dic = {}
    for i in range(8):
        data_content = nfile_lines[start_num + 8 * j + i] #
第j组, 第i行
        n_dic['数据组数'] = j + 1
        if i == 0:
            n_dic['卫星PRN号'] = str(data_content[0:3])
            n_dic['历元'] = data_content[4:23]
            n_dic['卫星钟偏差(s)'] =
float((data_content.strip('\n')[23:42])) # 利用字符串切片功能来进行字符串的修改
            n_dic['卫星钟漂移(s/s)'] =
float((data_content.strip('\n')[42:61]))
            n_dic['卫星钟漂移速度(s/s*s)'] =
float((data_content.strip('\n')[61:80]))
            if i == 1:
                n_dic['IODE'] =
float((data_content.strip('\n')[4:23]))
                n_dic['C_rs'] =
float((data_content.strip('\n')[23:42]))
                n_dic['n'] =
float((data_content.strip('\n')[42:61]))
                n_dic['M0'] =
float((data_content.strip('\n')[61:80]))
                if i == 2:
                    n_dic['C_uc'] =
float((data_content.strip('\n')[4:23]))
                    n_dic['e'] =
float((data_content.strip('\n')[23:42]))

```

```

        n_dic['C_us'] =
float((data_content.strip('\n')[42:61]))
        n_dic['sqrt_A'] =
float((data_content.strip('\n')[61:80]))
        if i == 3:
            n_dic['TEO'] =
float((data_content.strip('\n')[4:23]))
            n_dic['C_ic'] =
float((data_content.strip('\n')[23:42]))
            n_dic['OMEGA'] =
float((data_content.strip('\n')[42:61]))
            n_dic['C_is'] =
float((data_content.strip('\n')[61:80]))

        if i == 4:
            n_dic['I_0'] =
float((data_content.strip('\n')[4:23]))
            n_dic['C_rc'] =
float((data_content.strip('\n')[23:42]))
            n_dic['w'] =
float((data_content.strip('\n')[42:61]))
            n_dic['OMEGA_DOT'] =
float((data_content.strip('\n')[61:80]))
            if i == 5:
                n_dic['IDOT'] =
float((data_content.strip('\n')[4:23]))
                n_dic['L2_code'] =
float((data_content.strip('\n')[23:42]))
                n_dic['PS_week_num'] =
float((data_content.strip('\n')[42:61]))

            if i == 6:
                n_dic['卫星精度(m)'] =
float((data_content.strip('\n')[4:23]))
                n_dic['卫星健康状态'] =
float((data_content.strip('\n')[23:42]))
                n_dic['TGD'] =
float((data_content.strip('\n')[42:61]))
                n_dic['IODC'] =
float((data_content.strip('\n')[61:80]))
                n_dic_list.append(n_dic) # 存储字典

    with open('北斗卫星.csv', 'w', newline='') as f:
        header = ['数据组数', '卫星 PRN 号', '历元', '卫星钟偏差(s)',
'卫星钟漂移(s/s)', '卫星钟漂移速度(s/s*s)', 'IODE', 'C_rs', 'n',

```



```

'M0', 'C_uc', 'e', 'C_us', 'sqrt_A', 'TEO', 'C_ic', 'OMEGA',
'C_is', 'I_0', 'C_rc', 'w', 'OMEGA_DOT', 'IDOT', 'L2_code',
'PS_week_num', 'L2_P_code', '卫星精度(m)', '卫星健康状态',
'TGD', 'IODE', 'X', 'Y', 'Z']
    writer = csv.DictWriter(f, fieldnames=header) # 写入表头
    writer.writeheader() # 写入表头
    writer.writerows(n_dic_list) # 写入数据
f.close()

prn_x_y_z = [] # 存储卫星位置
with open('北斗卫星.csv', 'rt') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        PRN = str(row["卫星 PRN 号"])
        TIME = row["历元"]
        year = int(TIME.strip('\n')[2:4])
        month = int(TIME.strip('\n')[5:7])
        day = int(TIME.strip('\n')[8:10])
        hour = int(TIME.strip('\n')[11:13])
        minute = int(TIME.strip('\n')[14:16])
        second = float(TIME.strip('\n')[17:19])
        a_0 = float(row["卫星钟偏差(s)"])
        a_1 = float(row["卫星钟漂移(s/s)"])
        a_2 = float(row["卫星钟漂移速度(s/s*s)"])
        IODE = float(row["IODE"])
        C_rs = float(row["C_rs"])
         $\delta n$  = float(row["n"])
        M0 = float(row["M0"])
        C_uc = float(row["C_uc"])
        e = float(row["e"])
        C_us = float(row["C_us"])
        sqrt_A = float(row["sqrt_A"])
        TEO = float(row["TEO"])
        C_ic = float(row["C_ic"])
        OMEGA = float(row["OMEGA"])
        C_is = float(row["C_is"])
        I_0 = float(row["I_0"])
        C_rc = float(row["C_rc"])
        w = float(row["w"])
        OMEGA_DOT = float(row["OMEGA_DOT"])
        IDOT = float(row["IDOT"])
        L2_code = float(row["L2_code"])
        PS_week_num = float(row["PS_week_num"])
        TGD = float(row["TGD"])

```

```

        IODC = float(row["IODC"])
        t1 = None

        # 1.计算卫星运行平均角速度 GM:WGS84 下的引力常数
=3.986005e14, a:长半径
        GM = 398600500000000
        n_0 = m.sqrt(GM) / m.pow(sqrt_A, 3)
        n = n_0 +  $\delta n$ 

        # 2.计算归化时间 t_k 计算 t 时刻的卫星位置 UT: 世界时 此
        处以小时为单位
        UT = hour + (minute / 60.0) + (second / 3600)
        # GPS 时起始时刻 1980 年 1 月 6 日 0 点 year 是两位数 需要
        转换到四位数
        if year >= 80:
            if year == 80 and month == 1 and day < 6:
                year = year + 2000
            else:
                year = year + 1900
        else:
            year = year + 2000
        if month <= 2:
            year = year - 1
            month = month + 12 # 1, 2 月视为前一年 13, 14 月

        # 需要将当前需计算的时刻先转换到儒略日再转换到 GPS 时间
        JD = (365.25 * year) + int(30.6001 * (month + 1)) +
        day + UT / 24 + 1720981.5

        WN = int((JD - 2444244.5) / 7) # WN:GPS_week number
        目标时刻的 GPS 周

        t_oc = ((JD - 2444244.5) - (7.0 * WN)) * 24 * 3600.0
        - 14 # t_GPS:目标时刻的 GPS 秒 减去 14 秒为 BDT

        # 对观测时刻 t1 进行钟差改正,注意: t1 应是由接收机接收到的
        时间

        t_k = -14

        # 3.平近点角计算 M_k = M_0+n*t_k
        M_k = M0 + n * t_k # 实际应该是乘 t_k, 但是没有接收机的
        观测时间, 所以为了练手设 t_k=0
    
```

```

# 4.偏近点角计算 E_k (迭代计算)  $E_k = M_k + e \cdot \sin(E_k)$ 
E = 0
E1 = 1
count = 0
while abs(E1 - E) > 1e-10:
    count = count + 1
    E1 = E
    E = M_k + e * m.sin(E)
    if count > 1e8:
        print("计算偏近点角时未收敛！")
        break

# 5.计算卫星的真近点角
V_k = m.atan2((m.sqrt(1 - e * e) * m.sin(E)),
(m.cos(E) - e));

# 6.计算升交距角 u_0( $\phi_k$ ),  $\omega$ :卫星电文给出的近地点角距
u_0 = V_k + w

# 7.摄动改正项  $\delta u$ 、 $\delta r$ 、 $\delta i$  :升交距角 u、卫星矢径 r 和轨道倾角 i 的摄动量
 $\delta u = C_{uc} * m.\cos(2 * u_0) + C_{us} * m.\sin(2 * u_0)$ 
 $\delta r = C_{rc} * m.\cos(2 * u_0) + C_{rs} * m.\sin(2 * u_0)$ 
 $\delta i = C_{ic} * m.\cos(2 * u_0) + C_{is} * m.\sin(2 * u_0)$ 

# 8.计算经过摄动改正的升交距角 u_k、卫星矢径 r_k 和轨道倾角 i_k
u = u_0 +  $\delta u$ 
r = m.pow(sqrt_A, 2) * (1 - e * m.cos(E)) +  $\delta r$ 
i = I_0 +  $\delta i$  + IDOT * (t_k); # 实际乘 t_k=t-t_oe

# 9.计算卫星在轨道平面坐标系的坐标,卫星在轨道平面直角坐标系 (X 轴指向升交点) 中的坐标为:
x_k = r * m.cos(u)
y_k = r * m.sin(u)

# 10.观测时刻升交点经度  $\Omega_k$  的计算,升交点经度  $\Omega_k$  等于观测时刻升交点赤经  $\Omega$  与格林尼治恒星时 GAST 之差  $\Omega_k = \Omega_0 + (\omega_{DOT} - \omega_e) * t_k - \omega_e * t_{oe}$ 
omega_e = 7.292115e-5 # 地球自转角速度
OMEGA_k = OMEGA + (OMEGA_DOT - omega_e) * t_k - omega_e * TEO; # 星历中给出的 Omega 即为 Omega_o=Omega_t_oe-GAST_w

```

```

        # 11. 计算卫星在地固系中的直角坐标 1
        X_k = x_k * m.cos(OMEGA_k) - y_k * m.cos(i) *
m.sin(OMEGA_k)
        Y_k = x_k * m.sin(OMEGA_k) + y_k * m.cos(i) *
m.cos(OMEGA_k)
        Z_k = y_k * m.sin(i)

        # 12. 判断卫星是否为 GEO 卫星，否则不进行极移改正。
        if PRN in ['C01', 'C02', 'C03', 'C04', 'C05', 'C59',
'C60', 'C61']:

            fi = omega_e * t_k
            five = (m.pi / 180 * 5) * (-1)

            a = np.asmatrix([X_k, Y_k, Z_k])

            a = rz(fi) * rx(five) * a.T

            X_k = str(a[0, 0])
            Y_k = str(a[1, 0])
            Z_k = str(a[2, 0])

            if month > 12: # 恢复历元
                year = year + 1
                month = month - 12

            prn_x_y_z.append(PRN + ",")
            prn_x_y_z.append(str(X_k) + ",")
            prn_x_y_z.append(str(Y_k) + ",")
            prn_x_y_z.append(str(Z_k))
            prn_x_y_z.append("\n")

        else:
            prn_x_y_z.append(PRN + ",")
            prn_x_y_z.append(str(X_k) + ",")
            prn_x_y_z.append(str(Y_k) + ",")
            prn_x_y_z.append(str(Z_k))
            prn_x_y_z.append("\n")

    return prn_x_y_z
rnx_filename=[]
dir=os.listdir()
for i in range(len(dir)):
    line=dir[i]

```

```

        extension=line.split(".")
        if extension[len(extension)-1] in "rnx":
            rnx_filename.append(line)
num=1
for i in rnx_filename:
    f = open("北斗卫星位置.txt", 'w')
    f.writelines(reader_GNSS(i))
    print("文件写入成功！")
    f.close()
    num+=1
    print("完成计算的星历名称为: " + str(i))

```

draw2D.py:

```

import pandas as pd
import plotly.graph_objects as go

# 读取数据
data = pd.read_csv('北斗卫星位置.txt', header=None, names=['卫星名称', 'x', 'y', 'z'])

# 建立卫星编号与类型的映射关系
def get_satellite_type(satellite):
    if satellite in ['C01', 'C02', 'C03', 'C04', 'C05', 'C59', 'C60', 'C61']:
        return 'GEO'
    elif satellite in ['C06', 'C07', 'C08', 'C09', 'C10', 'C13', 'C16', 'C31', 'C38', 'C39', 'C40']:
        return 'IGSO'
    else:
        return 'MEO'

data['卫星类型'] = data['卫星名称'].apply(get_satellite_type)

# 创建平面散点图
fig = go.Figure()

# 存储所有卫星轨迹的可见性状态，初始都为 True
all_visible = []
# 定义不同类型卫星的颜色
type_colors = {'GEO': 'red', 'IGSO': 'orange', 'MEO': 'blue'}

# 存储每个卫星的可见性状态，初始全为 True

```



```

visibility_states = []
for satellite, group in data.groupby('卫星名称'):
    satellite_type = get_satellite_type(satellite)
    fig.add_trace(go.Scatter(
        x=group['x'],
        y=group['y'],
        mode='markers',
        name=satellite,
        marker=dict(
            size=5,
            opacity=0.8,
            color=type_colors[satellite_type]
        ),
    ))
    visibility_states.append(True)

# 生成每个卫星单独显示时的可见性列表
buttons = []

# 添加全部显示按钮
buttons.append({
    'label': '全部显示',
    'method': 'update',
    'args': [{ 'visible': visibility_states },
             { 'title': '卫星平面散点图' }]
})

# 按卫星类型添加显示按钮
satellite_types = data['卫星类型'].unique()
for sat_type in satellite_types:
    visibility = [get_satellite_type(fig.data[i].name) ==
sat_type for i in range(len(fig.data))]
    buttons.append({
        'label': sat_type,
        'method': 'update',
        'args': [{ 'visible': visibility },
                 { 'title': f'{sat_type} 卫星轨迹' }]
    })

for i in range(len(fig.data)):
    new_visibility = [False] * len(fig.data)
    new_visibility[i] = True
    buttons.append({
        'label': fig.data[i].name,
        'method': 'update',
    })

```

```

        'args': [{ 'visible': new_visibility},
                  { 'title': f'{fig.data[i].name} 卫星轨迹'}]
    })

# 更新布局，添加下拉菜单
fig.update_layout(
    title='卫星平面散点图',
    xaxis_title='x 坐标',
    yaxis_title='y 坐标',
    plot_bgcolor='white',
    paper_bgcolor='white',
    updatemenus=[{
        'buttons': buttons,
        'direction': 'down',
        'showactive': True,
        'x': 1.1,
        'xanchor': 'left',
        'y': 1.05,
        'yanchor': 'top'
    }]
)

fig.show()

#保存为 html 文件
fig.write_html('卫星平面散点图.html')

```

draw3D. py:

```

import pandas as pd
import plotly.graph_objects as go

# 读取数据
data = pd.read_csv('北斗卫星位置.txt', header=None, names=['卫星名称', 'x', 'y', 'z'])

# 建立卫星编号与类型的映射关系
def get_satellite_type(satellite):
    if satellite in ['C01', 'C02', 'C03', 'C04', 'C05', 'C59', 'C60', 'C61']:
        return 'GEO'
    elif satellite in ['C06', 'C07', 'C08', 'C09', 'C10', 'C13', 'C16', 'C31', 'C38', 'C39', 'C40']:

```

```

        return 'IGSO'
    else:
        return 'MEO'

data['卫星类型'] = data['卫星名称'].apply(get_satellite_type)

# 创建 3D 散点图
fig = go.Figure()

# 存储所有卫星轨迹的可见性状态，初始都为 True
all_visible = []
# 定义不同类型卫星的颜色
type_colors = {'GEO': 'red', 'IGSO': 'orange', 'MEO': 'blue'}

# 为每个卫星添加轨迹
for satellite, group in data.groupby('卫星名称'):
    satellite_type = get_satellite_type(satellite)
    fig.add_trace(go.Scatter3d(
        x=group['x'],
        y=group['y'],
        z=group['z'],
        mode='markers',
        name=satellite,
        marker=dict(
            size=5,
            opacity=0.8,
            color=type_colors[satellite_type]
        )
    ))
    all_visible.append(True)

# 生成每个卫星单独显示时的可见性列表
buttons = []

# 添加全部显示按钮
buttons.append({
    'label': '全部显示',
    'method': 'update',
    'args': [{'visible': all_visible},
             {'title': '卫星立体散点图'}]
})

# 按卫星类型添加显示按钮
satellite_types = data['卫星类型'].unique()

```

```

for sat_type in satellite_types:
    visibility = [get_satellite_type(fig.data[i].name) ==
sat_type for i in range(len(fig.data))]
    buttons.append({
        'label': sat_type,
        'method': 'update',
        'args': [{'visible': visibility},
                {'title': f'{sat_type} 卫星轨迹'}]
    })

for i in range(len(fig.data)):
    visibility = [False] * len(fig.data)
    visibility[i] = True
    buttons.append({
        'label': fig.data[i].name,
        'method': 'update',
        'args': [{'visible': visibility},
                {'title': f'{fig.data[i].name} 卫星轨迹'}]
    })

# 更新布局，添加下拉菜单
fig.update_layout(
    scene=dict(
        xaxis_title='x 坐标',
        yaxis_title='y 坐标',
        zaxis_title='z 坐标'
    ),
    title='卫星立体散点图',
    updatemenus=[{
        'buttons': buttons,
        'direction': 'down',
        'showactive': True,
        'x': 1.1,
        'xanchor': 'left',
        'y': 1.05,
        'yanchor': 'top'
    }]
)

fig.show()

#保存为 html 文件
fig.write_html("卫星立体散点图.html")

```

sel6-45.py:

```
# 定义输入文件路径
input_file_path = '北斗卫星位置.txt'
# 定义输出文件路径
output_file_path = '北斗卫星位置_C06-C45.txt'

# 打开输入文件和输出文件
with open(input_file_path, 'r', encoding='utf-8') as input_file, \
    open(output_file_path, 'w', encoding='utf-8') as output_file:
    # 遍历输入文件的每一行
    for line in input_file:
        # 去除行首尾的空白字符
        line = line.strip()
        # 遍历 C06 到 C45 的卫星编号
        for i in range(6, 46):
            satellite_id = f'C{i:02d}'
            # 如果行以当前卫星编号开头
            if line.startswith(satellite_id):
                # 按逗号分割行数据
                parts = line.split(',')
                # 提取卫星编号
                new_parts = [parts[0]]
                # 遍历剩余的数据部分
                for part in parts[1:]:
                    try:
                        # 将数据转换为浮点数并除以 1000
                        new_value = float(part) / 1000
                        # 将结果添加到新的数据部分列表
                        new_parts.append(str(new_value))
                    except ValueError:
                        # 如果转换失败, 保持原始数据
                        new_parts.append(part)
                # 用逗号重新连接新的数据部分
                new_line = ','.join(new_parts)
                # 将新行写入输出文件
                output_file.write(new_line + '\n')
            break
```

sp3.py:

```
# 定义文件路径
file_path = 'WUM0MGXRAP_20250550000_01D_05M_ORB.SP3'
# 用于存储筛选后的数据
filtered_data = []
# 标记是否处于有效的时间行之后
in_valid_time_section = False

# 打开文件并逐行读取
with open(file_path, 'r', encoding='utf-8') as file:
    for line in file:
        # 检查是否为时间行
        if line.startswith('*'):
            # 解析时间信息
            parts = line.split()
            if len(parts) >= 7 and parts[1] == '2025' and
parts[2] == '2' and parts[3] == '24' and parts[5] == '0':
                # 标记为处于有效时间行之后
                in_valid_time_section = True
            else:
                in_valid_time_section = False
        # 检查是否为 PC 开头的卫星数据，并且处于有效时间行之后
        elif line.startswith('PC') and in_valid_time_section:
            # 删除最后一列数据
            parts = line.strip().split()[:-1]
            # 去掉首字母 P
            if parts[0].startswith('P'):
                parts[0] = parts[0][1:]
            # 使用逗号分隔各列
            new_line = ','.join(parts)
            # 将符合条件的数据添加到筛选结果中
            filtered_data.append(new_line)

# 仅对第一列进行增序排列
filtered_data.sort(key=lambda x: x.split(',')[0])

# 你也可以将筛选后的数据保存到新文件中
output_file_path = 'sp3_data.txt'
with open(output_file_path, 'w', encoding='utf-8') as
output_file:
    for data in filtered_data:
        output_file.write(data + '\n')
```

ana.py:

```
# 读取文件内容
def read_file(file_path):
    data = {}
    with open(file_path, 'r') as file:
        for line in file:
            parts = line.strip().split(',')
            satellite = parts[0]
            if satellite not in data:
                data[satellite] = []
            data[satellite].append([float(part) for part in
parts[1:]])
    return data

# 计算差值和百分差
def calculate_differences(data1, data2):
    differences = []
    percentage_differences = []
    for satellite in data1:
        if satellite in data2:
            for i in range(len(data1[satellite])):
                diff = [(data1[satellite][i][j] -
data2[satellite][i][j]) * 1000 for j in range(3)]
                pct_diff = [(data1[satellite][i][j] -
data2[satellite][i][j]) / data2[satellite][i][j] * 100 if
data2[satellite][i][j] != 0 else 0
for j in range(3)]
                diff = [round(abs(d), 7) for d in diff] # 取绝对
值
                pct_diff = [round(abs(p), 7) for p in
pct_diff] # 取绝对值
                differences.append([satellite] + diff)
                percentage_differences.append([satellite] +
pct_diff)
    return differences, percentage_differences

# 将百分差转换为科学计数法并保留 7 位有效数字
def to_scientific_notation(data):
    new_data = []
    for row in data:
        satellite = row[0]
        values = [format(float(num), '.7e') for num in
row[1:]] # 转换全部百分差数据
        new_data.append([satellite] + values)
    return new_data
```



```

# 写入新文件
def write_to_file(differences, percentage_differences,
output_path):
    with open(output_path, 'w') as file:
        for i in range(len(differences)):
            diff_str = ','.join([str(d) for d in differences[i]])
            pct_diff =
to_scientific_notation([percentage_differences[i]])[0]
            pct_diff_str = ','.join(pct_diff[1:]) # 去掉卫星编号
            file.write(f"{diff_str},{pct_diff_str}\n")
if __name__ == "__main__":
    file1_path = '北斗卫星位置_C06-C45.txt'
    file2_path = 'sp3_data.txt'
    output_path = 'ana.txt'

    data1 = read_file(file1_path)
    data2 = read_file(file2_path)

    differences, percentage_differences =
calculate_differences(data1, data2)
    write_to_file(differences, percentage_differences,
output_path)

```

config.py:

```

def calculate_column_average(file_path):
    data = []
    with open(file_path, 'r') as file:
        for line in file:
            line = line.strip()
            if line:
                values = list(map(float, line.split(',')[1:]))
                data.append(values)

    num_columns = len(data[0])
    column_sums = [0] * num_columns

    for row in data:
        for i in range(num_columns):
            column_sums[i] += abs(row[i])

```

```

        column_averages = [sum_value / len(data) for sum_value in
column_sums]

    return column_averages

def write_to_config(averages, output_path):
    with open(output_path, 'w') as file:
        headers = ["x 坐标差值平均值", "y 坐标差值平均值", "z 坐标差值
平均值",
                    "x 坐标百分差平均值", "y 坐标百分差平均值", "z 坐标
百分差平均值"]
        for header, average in zip(headers, averages):
            file.write(f"{header}: {average}\n")

if __name__ == "__main__":
    input_file = "ana.txt"
    output_file = "config.txt"
    averages = calculate_column_average(input_file)
    write_to_config(averages, output_file)

```

plot.py:

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

def plot_boxplots(file_path):
    # 读取数据到 DataFrame
    df = pd.read_csv(file_path, header=None, names=['id', 'x',
'y', 'z', 'x_percent', 'y_percent', 'z_percent'])

    # 提取差值数据列
    diff_columns = ['x', 'y', 'z']
    diff_data = [df[col] for col in diff_columns]

    # 提取百分差数据列
    percent_diff_columns = ['x_percent', 'y_percent',
'z_percent']
    percent_diff_data = [df[col] for col in percent_diff_columns]

    # 计算差值数据列的绝对值平均值

```

```

    diff_abs_mean = [np.mean(np.abs(df[col])) for col in
diff_columns]

    # 计算百分差数据列的绝对值平均值
    percent_diff_abs_mean = [np.mean(np.abs(df[col])) for col in
percent_diff_columns]

    # 绘制差值数据的箱线图
    plt.figure(figsize=(10, 6))
    positions_diff = np.arange(len(diff_columns))
    bp_diff = plt.boxplot(diff_data, positions=positions_diff,
widths=0.5)
    plt.xticks(positions_diff, diff_columns)
    plt.xlabel('Coordinate Difference Columns')
    plt.ylabel('Values')
    plt.title('Boxplots of Coordinate Differences')

    for i, box in enumerate(bp_diff['boxes']):
        y = box.get_ydata()[1]
        plt.text(positions_diff[i], y, f'{diff_abs_mean[i]:.4f}',
ha='center', va='bottom')
    plt.grid(True, axis='y', linestyle='--', alpha=0.7)
    plt.show()

if __name__ == "__main__":
    input_file = "ana.txt"
    plot_boxplots(input_file)

    #保存图片
    plt.savefig("boxplot.png")

```

plot1.py:

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

def plot_scatter_with_mean(file_path):
    # 读取数据到 DataFrame
    df = pd.read_csv(file_path, header=None, names=['id', 'x',
'y', 'z', 'x_percent', 'y_percent', 'z_percent'])

    # 计算 x、y、z 差值的绝对值平均值

```

```
x_mean = np.mean(np.abs(df['x']))
y_mean = np.mean(np.abs(df['y']))
z_mean = np.mean(np.abs(df['z']))

# 创建子图
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 绘制 x 差值的散点图, 纵坐标使用绝对值
axes[0].scatter(range(len(df['x'])), np.abs(df['x']))
axes[0].axhline(y=x_mean, color='r', linestyle='--',
label=f'Abs Mean: {x_mean:.4f}')
axes[0].set_title('X Difference Scatter Plot')
axes[0].set_xlabel('Data Point Index')
axes[0].set_ylabel('Absolute X Difference Value')
axes[0].legend()

# 绘制 y 差值的散点图, 纵坐标使用绝对值
axes[1].scatter(range(len(df['y'])), np.abs(df['y']))
axes[1].axhline(y=y_mean, color='r', linestyle='--',
label=f'Abs Mean: {y_mean:.4f}')
axes[1].set_title('Y Difference Scatter Plot')
axes[1].set_xlabel('Data Point Index')
axes[1].set_ylabel('Absolute Y Difference Value')
axes[1].legend()

# 绘制 z 差值的散点图, 纵坐标使用绝对值
axes[2].scatter(range(len(df['z'])), np.abs(df['z']))
axes[2].axhline(y=z_mean, color='r', linestyle='--',
label=f'Abs Mean: {z_mean:.4f}')
axes[2].set_title('Z Difference Scatter Plot')
axes[2].set_xlabel('Data Point Index')
axes[2].set_ylabel('Absolute Z Difference Value')
axes[2].legend()

plt.tight_layout()
plt.show()
if __name__ == "__main__":
    input_file = "ana.txt"

    plot_scatter_with_mean(input_file)

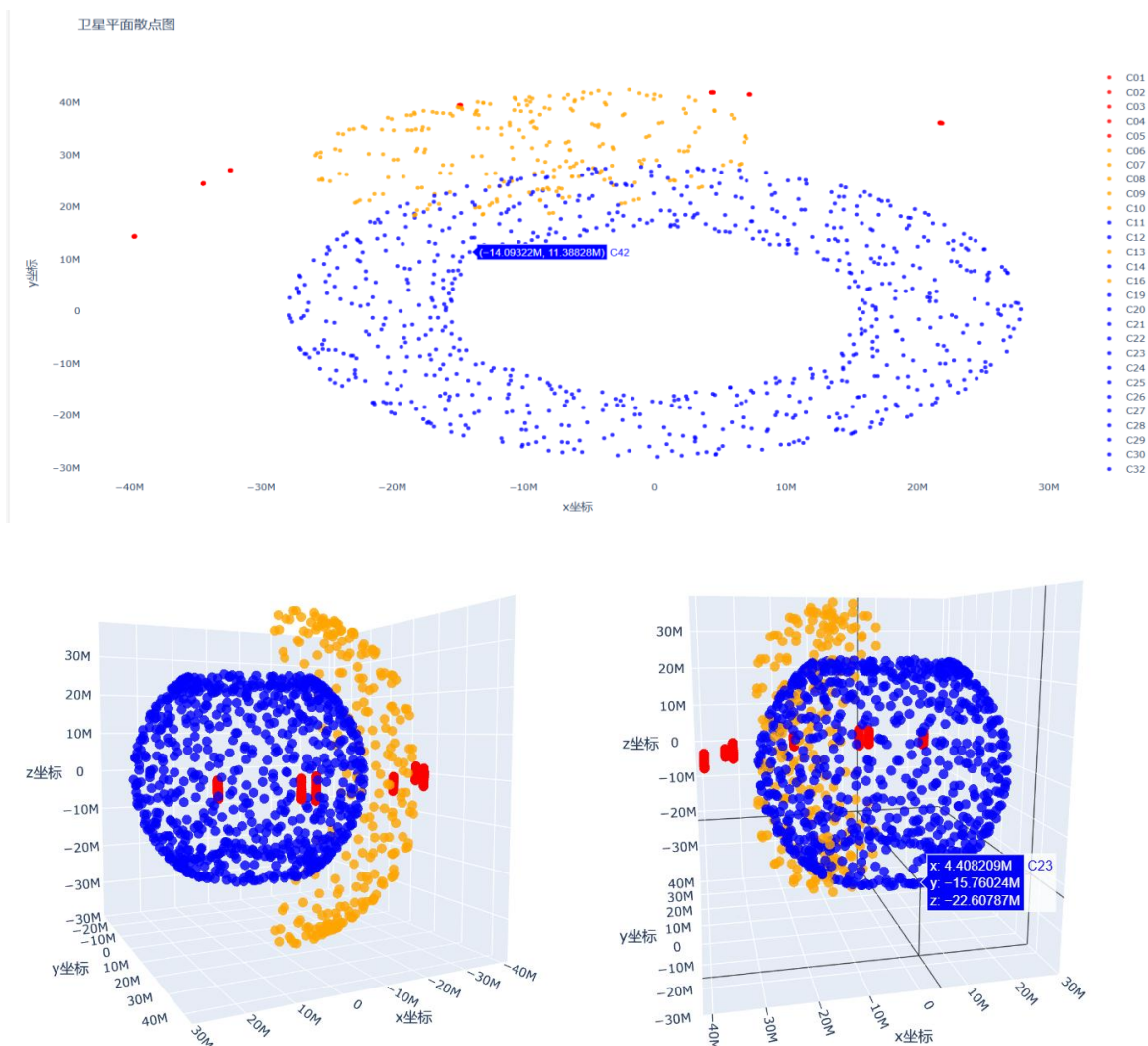
# 保存到文件
plt.savefig("point.png")
```

三、 结果

1. 卫星坐标位置计算结果 北斗卫星位置.txt:

```
C01,-34317297.48966522,24448608.32179622,-719708.2920459607
C01,-34320853.360554904,24434811.337615788,-891777.6855465199
C01,-34327666.62107457,24420862.612070087,-1002904.7500512585
C01,-34337351.328319654,24408394.125994053,-1045450.1407520097
C01,-34348985.41109106,24398883.31562856,-1016486.9986681767
C01,-34361242.97531328,24393383.560267344,-918002.5278794274
C01,-34372626.75877813,24392342.03098064,-756758.1891413368
C01,-34381747.40779633,24395547.180997472,-543819.1522666288
C01,-34387584.18986349,24402214.59224401,-293788.7398556513
```

2. 卫星坐标图绘制结果（红色 GEO，橙色 IGSO，蓝色 MEO）：



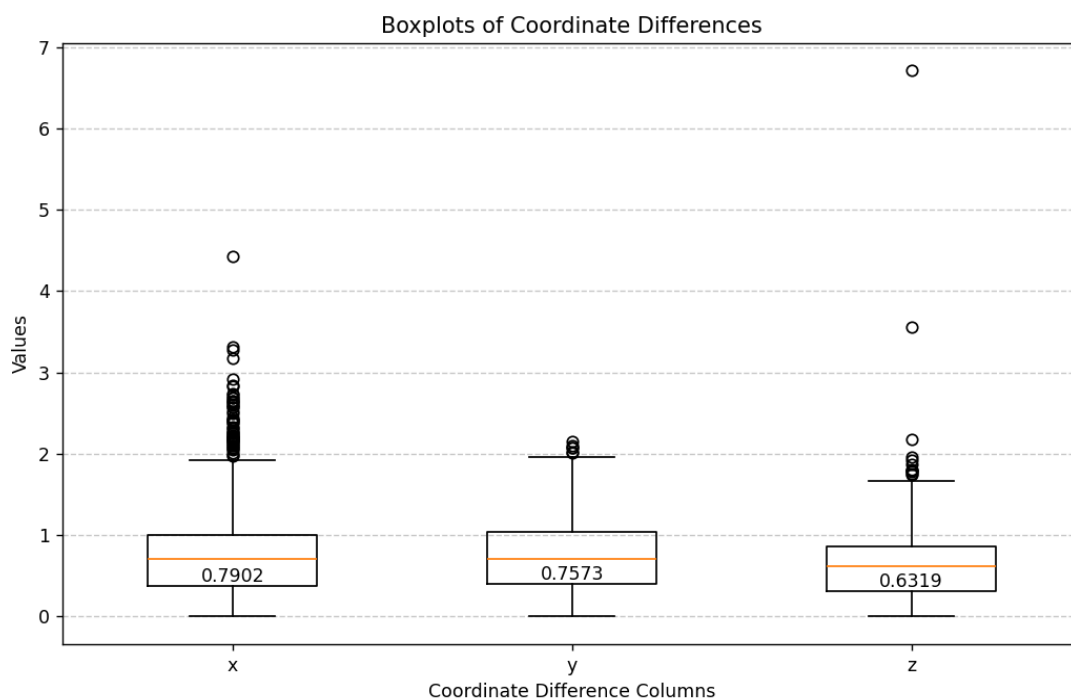
3. 误差及百分差计算结果（单位：m） ana.txt:

```
C06,2.6485135,0.2020988,0.9443663,3.6300000e-05,9.0000000e-07,2.8000000e-06
C06,2.5682412,0.1366622,0.6827367,2.1800000e-05,6.0000000e-07,2.1000000e-06
C06,2.4178459,0.1228665,0.4242365,1.5500000e-05,5.0000000e-07,1.5000000e-06
C06,2.4503139,0.1035207,0.3013615,1.4000000e-05,3.0000000e-07,1.3000000e-06
C06,2.4213804,0.2444497,0.4538525,1.4100000e-05,7.0000000e-07,2.9000000e-06
C06,2.6863602,0.407055,0.6426873,1.8300000e-05,1.1000000e-06,8.8000000e-06
C06,3.3111677,0.8683138,0.321907,3.1300000e-05,2.1000000e-06,1.9400000e-05
C06,3.2746066,1.1673479,0.3257115,5.3600000e-05,2.9000000e-06,3.1000000e-06
C06,2.8467665,1.3243778,0.4488983,1.1590000e-04,3.5000000e-06,2.4000000e-06
C06,2.4923461,1.3443831,0.4860727,3.9920000e-04,4.0000000e-06,1.9000000e-06
C06,2.1014052,1.2923594,0.4834969,1.8620000e-04,4.5000000e-06,1.6000000e-06
C06,1.701399,1.0409984,0.5337234,4.4300000e-05,4.1000000e-06,1.6000000e-06
C06,1.5981482,0.7172676,0.5106554,1.9900000e-05,3.1000000e-06,1.5000000e-06
C06,1.6185504,0.2927911,0.4460661,1.2800000e-05,1.2000000e-06,1.4000000e-06
```

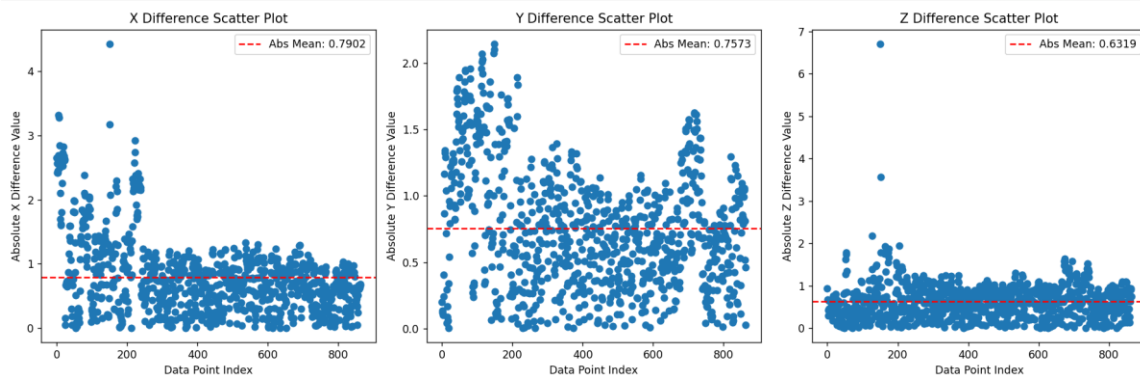
4. 整体计算精度评定（单位：m） config.txt:

```
x坐标差值平均值: 0.7901932350694444
y坐标差值平均值: 0.7572539730324068
z坐标差值平均值: 0.6318650188657409
x坐标百分差平均值: 1.629803240740741e-05
y坐标百分差平均值: 6.437152777777767e-06
z坐标百分差平均值: 1.145474537037036e-05
```

绘制各卫星 x 轴, y 轴, z 轴误差的箱线图。从图中可以看出: 各轴数据的最大误差均小于 2m, 最小误差接近 0m, 三轴平均误差分别为 0.7902m, 0.7573m, 0.6319m, 存在一定的异常值。



绘制各卫星 x 轴，y 轴，z 轴误差的散点分布图。从图中可以看出：y 轴的计算误差整体不存在极端值，且编号在 C06-C15 的数据计算误差较大，其余编号的卫星的误差分布比较理想。



四、 总结

本次课后作业利用卫星广播星历计算出了卫星的位置计算值，并与真值进行了误差计算和进度评估。首先，从指定网站下载广播星历文件，经数据预处理，提取北斗卫星数据并解析存入字典，再依据一系列公式计算卫星坐标位置，输出结果文件并绘制 2D、3D 图。同时，对精密星历数据预处理，与计算值对比计算误差和百分差，绘制误差散点图和箱线图评估精度。

从结果来看，成功计算出卫星坐标位置，绘制出卫星坐标图，清晰展示不同类型卫星分布。误差及百分差计算结果表明，不同卫星在各坐标方向上存在一定误差。整体计算精度评定显示，x、y、z 坐标差值平均值分别为 0.7901932350694444m、0.7572539730324068m、0.6318650188657409m，百分差平均值也处于 10^{-5} 数量级。此次作业完整实现了从数据处理到结果分析的流程。

通过这次作业，在知识层面，我深入理解了北斗卫星广播星历的构成和应用，以及卫星轨道计算的原理和方法，对卫星导航定位系统的运行机制有了更清晰的认知。在计算卫星位置时，部分公式理解困难，迭代计算偏近点角时还遇到未收敛的问题，花费不少时间排查解决。这次作业也让我认识到卫星导航定位领域的严谨性和复杂性，每个数据、每个步骤都至关重要。